

Mock School System - Attacks and Countermeasures Life Cycle

SQL Injection Attack

Research Assistant & Author Brian Alexis Almaguer

Principle Investigator Dr. Jun Dai

Background

You have access to and manage a school system and website to track students, teachers, and grades. This web based application hosts information that is considered Personally Identifying Information (PII) that can be accessed by a one-to-one (1:1) relationship, respectively, by the owner of the information; Teachers and Administrators, however, have a one-to-many (1:*) relationship to the data since they manage the records and there updates. The Web application is a three-layer application; it is coded in PHP, severed to the web using Apache, and uses MySQL to store the user data. The site has a log-in that creates one of two 1:1 or 1:* views for the data. The 1:* view allows for the data to be edited in an editing view. All of this is designed and needed by the school whose system you manage.

Scenario

It has been reported that a student was caught logging into a teachers account on the web applications. You received the reproduction steps and need to properly review the application to determine if this is a problem with the current version of the technology or the code of the web application; chances are it might be both, but you will not know until you workout the countermeasure and not a bandied solution.

SSDLC

The Secure Software Development Life Cycle (SSDLC), is what we are going to follow to since it helps organizations develop secure software. SSDLC is also what we are going to follow, since software, made with SSDLC, is developed with security considerations at every stage of the development process. More can be read here¹ in the wonderful Hackerone article.

Planning

1. Step 1 - Recreate the bug or attack

In the planning phase of the SSDLC, since we already have a preexisting product, we are going to need to recreate the bug or, in this case, the SQL Injection attack. The following is the SQL statement that should log in as a user.

```
1 $sql = "SELECT * FROM Authenticate WHERE userID='$username' AND userPW='$password' AND userLevel='$userLevel'";
```

The reported string used is the following.

```
1 '+(SELECT userPW FROM Authenticate WHERE userID='teacher')+'
```

To recreate the attack, all one needs to do is to know the teacher username and use the string above as the password.

2. Step 2 - Risk Assessment

Not all attacks/bugs actually need a Risk Assessment, since a large majority of them have been categorized on site such as OWASP² (Open Worldwide Application Security Project). In this case, since we have a reported example of abuse, we can search for the attack using the OWASP catalog.

¹<https://www.hackerone.com/knowledge-center/what-ssdlc-secure-software-development-life-cycle>

²<https://owasp.org/about/>

3. Step 3 - Mitigation

Since the Attack Recreation and Risk Assessment defined exactly what we are dealing with, we now need to research and catalog what is out there for mitigation's. A great starting point is to look at articles that OWASP has exactly like following³.

Requirements and Analysis

1. Step 1 - Analysis

Once the research has been completed on the attack and its recreation, the underlying technology or code needs to be connected to your work. Doing this makes the next step easier to do.

2. Step 2 - Requirements

This step can be fleshed out until the connection between the attack/bug is connected to the technology or code. That connection essentially writes your requirements out for you; it is your job to expand on those connections into simpler explanations and ideas.

For our specific example project, our problem originates in multiple places, leading to the following list of requirements:

- (a) The data submitted needs to be scrubbed since it is used immediately from the form.
- (b) Passwords need to have requirements that are enforced on the client side, since nothing is in place to prevent a user from submitting a bad login request.
- (c) The SQL statement should be more thorough to prevent extra embedded SQL statements.

Design and Prototyping

1. Step 1 - Design

Not all designs need to be implemented in the product to work during this phase. Some designs can be generated at a high level without actually writing code. In our case, We need to update not only our SQL Statement but also our front end. Below is an example of designing a variables that should pre-scrub our content

```
1 // Variables from the previous page coming from the POST method
2 $username = mysql_real_escape_string($_POST['myusername']);
3 $password = mysql_real_escape_string($_POST['mypassword']);
```

2. Step 2 - Prototyping

Like Design, prototyping does not need to be implemented in the product; prototypes can be made outside of the larger product to show proof of designs in that way the product does not receive any unguided development changes.

As an example prototype, below is an example of what the mysql_real_escape_string fuctions should do.

The Prototype

```
1 $example = "'+(SELECT userPW FROM Authenticate WHERE userID='teacher')+";
2 $example = mysql_real_escape_string($example);
3 echo 'Result: "$example<br>";
```

The Result

```
1 Result "'\'+(SELECT userPW FROM Authenticate WHERE userID=\'teacher\')+\'"
```

Development

1. Step 1 - Version Control or Backups

In Development, the very first thing you do is back up or version control your original program and settings before testing your prototypes on the product. Data should also be backed up just in case of a full system breakdown.

³https://owasp.org/www-community/attacks/SQL_Injection/

2. Step 2 - Run/Create a Non-Production Environment

Part of development is testing; true product testing can be done when you have a mirrored live or local product. In our case, we have a code ready to be edited live to see the change made live.

3. Step 3 - Integration

This step involves or effort, as you now have to integrate your design and prototype into the mock product before attempting a simulated attack. This step can happen multiple times over multiple days if we are going to make a countermeasure for a larger or more complicated attack.

Deployment

Deployment is the most important step as a maintainer since your web application will have to be down preventing any of your uses from accessing the data they need. Depending on the strategy used in the organization, a change process would be needed before making any changes. In the end, the process always follows an unfreeze-change-freeze flow.

Maintenance

Common regular web application maintenance should include performing daily backups of both application code and database content, monitoring server logs for errors or suspicious activities, and periodically updating the security of all dependencies and frameworks. In this scenario, it is imperative that backups of the code are saved.